Spring Cloud Openfeign

主要内容

Openfeign 简介
Openfeign 的请求参数处理
Openfeign 的性能优化
配置 Openfeign 负载均衡请求超时时间

学习目标

知识点	要求
Openfeign 简介	了解
Openfeign 的请求参数处理	掌握
Openfeign 的性能优化	掌握
配置 Openfeign 负载均衡请求超时时间	掌握

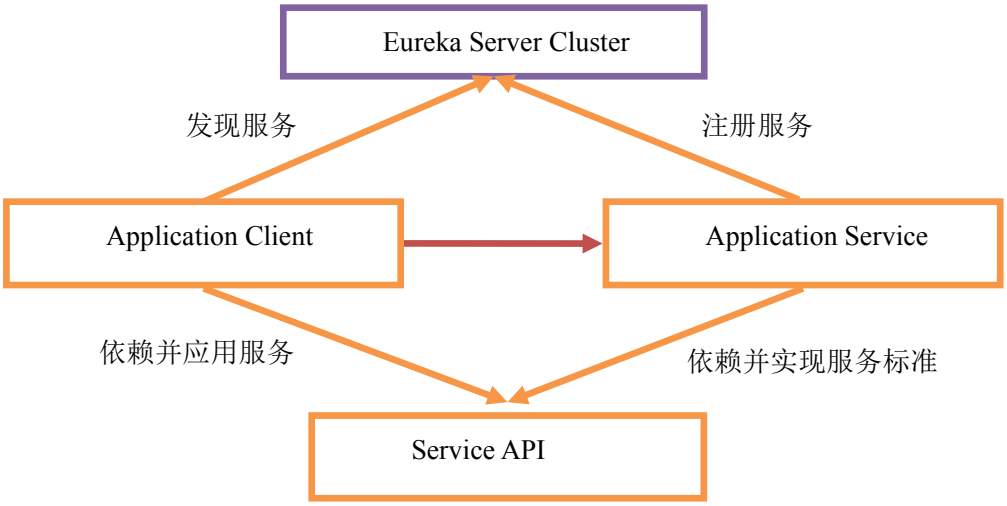
Spring Cloud Openfeign

一、 Openfeign 简介

Openfeign 是一种声明式、模板化的 HTTP 客户端(仅在 Application Client 中使用)。声明式调用是指，就像调用本地方法一样调用远程方法，无需感知操作远程 http 请求。

Spring Cloud 的声明式调用, 可以做到使用 HTTP 请求远程服务时能就像调用本地方法一样的体验，开发者完全感知不到这是远程方法，更感知不到这是个 HTTP 请求。Openfeign 的应用，让 Spring Cloud 微服务调用像 Dubbo 一样，Application Client 直接通过接口方法远程调用 Application Service，而不需要通过常规的 RestTemplate 构造请求再解析返回数据。它解决了让开发者调用远程接口就跟调用本地方法一样，无需关注与远程的交互细节，更无需关注分布式环境开发。

1 使用 Feign 技术开发时的应用部署结构



在使用 Openfeign 技术开发 Spring Cloud 微服务时，需要先抽取要注册发布的服务标准，将这套标准通过接口的形式定义出来。

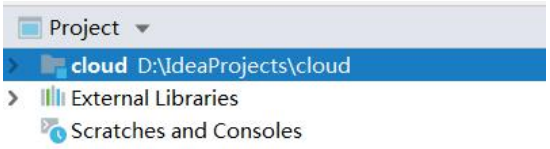
在 Application Service 端开发中，依赖抽取的服务标准接口工程，并对接口给予实现。

在 Application Client 端开发中，依赖抽取的服务标准接口工程，并应用接口信息和 Openfeign 技术，实现远程服务的调用。

在整体微服务开发中，Eureka Server 作为注册中心必不可少，注册中心的作用不变，仍旧是注册和发现服务。

二、 Openfeign 的请求参数处理

1 创建 Eureka Client 工程

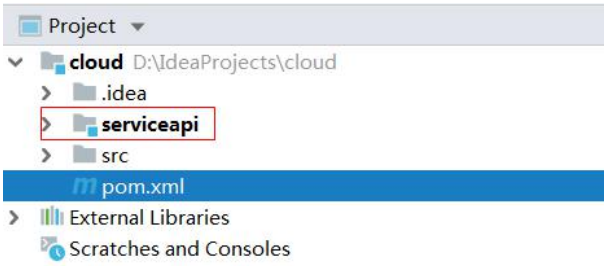


1.1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

```
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <packaging>pom</packaging>
    <modules>
        <module>serviceapi</module>
    </modules>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.2.2.RELEASE</version>
    </parent>
    <groupId>com.bjsxt</groupId>
    <artifactId>cloud</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
</project>
```

2 创建 Service API 服务标准 Module



3 开发服务标准 - Service API

3.1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloud</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>serviceapi</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
    </dependencies>

</project>
```

3.2 测试自定义参数类型

```
package com.bjsxt.entity;

import java.io.Serializable;
import java.util.Objects;

public class User implements Serializable {
    private Integer id;
    private String username;
    private String remark;

    public User(){}

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        User user = (User) o;
        return Objects.equals(id, user.id) &&
                Objects.equals(username, user.username) &&
                Objects.equals(remark, user.remark);
```

```
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, username, remark);
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getRemark() {
        return remark;
    }

    public void setRemark(String remark) {
        this.remark = remark;
    }
}
```

3.3 服务接口定义

```
package com.bjsxt.serviceapi;

import com.bjsxt.entity.User;
import org.springframework.web.bind.annotation.*;

import java.util.List;

public interface FirstServiceAPI {
    /**
```

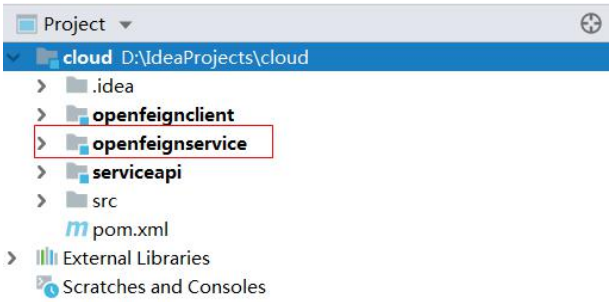
```
     * 测试 GET 请求的方法。
     * 请求不传递任何的参数。
     */
    @RequestMapping(value="/test", method= RequestMethod.GET)
    public List<String> testFeign();

    /**
     * 测试 GET 请求传递一个普通的参数。   /get?id=xxx&name=yyy
     * 在为 Feign 定义服务标准接口的时候，处理请求参数的方法参数，必须使用
@RequestParam 注解描述。
     * 且，无论方法参数名和请求参数名是否一致，都需要定义@RequestParam 注解的
value/name 属性。
     */
    @GetMapping(value="/get")
    public User getMultiParams(@RequestParam(value = "id") Integer id,
@RequestParam("name") String name);

    /**
     * 测试使用 POST 请求传递普通参数
     * 如果使用 POST 方式发起请求，传递多个普通参数，是使用请求头传递的参数。可
以使用@RequestParam 注解来处理请求参数。
     */
    @PostMapping(value="/post")
    public User postMultiParams(@RequestParam(value = "id") Integer id,
@RequestParam("name") String name);

    /**
     * 使用 POST 请求传递自定义类参数。
     * 必须使用@RequestBody 处理。
     */
    @PostMapping(value="/postObjectParam")
    public User postObjectParam(@RequestBody User pojo);
}
```

4 创建 Application Service 服务提供者 Module



5 开发服务提供者 - Application Service

5.1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloud</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>openfeignservice</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>com.bjsxt</groupId>
            <artifactId>serviceapi</artifactId>
            <version>1.0-SNAPSHOT</version>
        </dependency>
```

```
    </dependencies>

</project>
```

5.2 服务标准实现

```
package com.bjsxt.controller;

import com.bjsxt.entity.User;
import com.bjsxt.serviceapi.FirstServiceAPI;
import org.springframework.web.bind.annotation.RestController;

import java.util.Arrays;
import java.util.List;

/**
 * 对外提供服务的 Application Service。
 * 不能随便的定义服务。如果想让 Application Client 可以通过 Openfeign 技术访问当
前类型提供的服务，
 * 则必须遵循服务标准 - Service API
 */
@RestController
public class OpenfeignServiceController implements FirstServiceAPI {
    @Override
    public List<String> testFeign() {
        return Arrays.asList("测试 Openfeign", "此为返回结果");
    }

    @Override
    public User getMultiParams(Integer id, String name) {
        System.out.println("getMultiParams method run, parameters is [ id : " +
id + " ; name : " + name + " ]");
        User user = new User();
        user.setId(id);
        user.setUsername(name);
        return user;
    }

    @Override
    public User postMultiParams(Integer id, String name) {
        System.out.println("postMultiParams method run, parameters is [ id : "
+ id + " ; name : " + name + " ]");
        User user = new User();
```

```
        user.setId(id);
        user.setUsername(name);
        return user;
    }

    @Override
    public User postObjectParam(User pojo) {
        System.out.println("postObjectParam method run, parameters is [ " + pojo
+ " ]");
        return pojo;
    }
}
```

5.3 配置文件 application.yml

```
server:
  port: 8081

spring:
  application:
    name: openfeign-service

eureka:
  client:
    service-url:
      - http://localhost:8761/eureka/
```

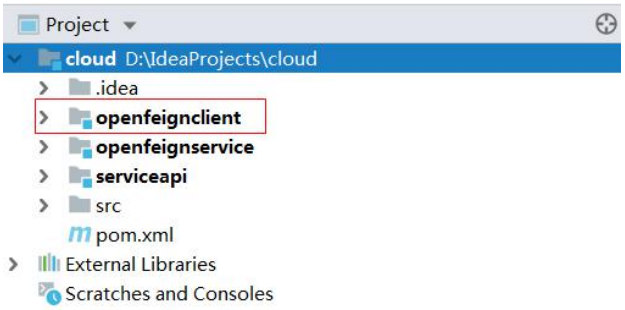
5.4 启动类

```
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class OpenfeignServiceApp {
    public static void main(String[] args) {
        SpringApplication.run(OpenfeignServiceApp.class, args);
    }
}
```

6 创建 Application Client 服务消费者 Module



7 开发服务消费者 - Application Client

7.1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloud</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>openfeignclient</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-openfeign</artifactId>
        </dependency>
        <dependency>
```

```
            <groupId>com.bjsxt</groupId>
            <artifactId>serviceapi</artifactId>
            <version>1.0-SNAPSHOT</version>
        </dependency>
    </dependencies>

</project>
```

7.2 本地服务接口

```
package com.bjsxt.service;

import com.bjsxt.serviceapi.FirstServiceAPI;
import org.springframework.cloud.openfeign.FeignClient;

/**
 * 本地接口，继承服务标准接口。
 * 在接口上增加注解@FeignClient，代表当前接口是一个 Openfeign 技术中的服务消费端。
 *  属性 name|value - 代表当前的 FeignClient 在请求 application service 的时候，是访问哪一个服务
 * 所谓的哪一个服务，就是 application service 全局配置文件中的 spring.application.name 属性值。
 */
@FeignClient("openfeign-service")
public interface FirstClientService extends FirstServiceAPI {
}
```

7.3 控制器开发

```
package com.bjsxt.controller;

import com.bjsxt.entity.User;
import com.bjsxt.service.FirstClientService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class FirstClientController {
```

```
    @Autowired
    private FirstClientService firstClientService;

    @GetMapping(value="/test")
    public List<String> testFeign() {
        return this.firstClientService.testFeign();
    }

    @GetMapping(value="/get")
    public User getMultiParams(Integer id, String name) {
        return this.firstClientService.getMultiParams(id, name);
    }

    @GetMapping(value="/post")
    public User postMultiParams(Integer id, String name) {
        return this.firstClientService.postMultiParams(id, name);
    }

    @GetMapping(value="/postObjectParam")
    public User postObjectParam(User pojo) {
        return this.firstClientService.postObjectParam(pojo);
    }
}
```

7.4 配置文件 application.yml

```
server:
  port: 8082

spring:
  application:
    name: openfeign-client

eureka:
  client:
    service-url:
      - http://localhost:8761/eureka/
```

7.5 启动类

```
package com.bjsxt;
```

```java
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;

/**
 * @EnableFeignClients - 描述当前应用是一个使用 Openfeign 技术开发的服务消费端。
 *  属性 backPackage - 扫描的包，即使用@FeignClient 描述的接口所在包。此属性可省略，默认扫描当前工程所有包。
 */
@SpringBootApplication
@EnableFeignClients(basePackages = "com.bjsxt.service")
public class OpenfeignClientApp {
    public static void main(String[] args) {
        SpringApplication.run(OpenfeignClientApp.class, args);
    }
}
```

8 参数处理简单总结

在 Openfeign 处理远程服务调用时，传递参数是通过 HTTP 协议传递的，参数存在的位置是请求头或请求体中。请求头传递的参数必须依赖@RequestParam 注解来处理请求参数，请求体传递的参数必须依赖@RequestBody 注解来处理请求参数。

三、 Openfeign 的性能优化

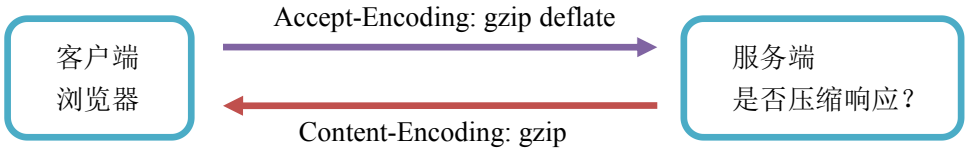
1 GZIP 简介

gzip 介绍：gzip 是一种数据格式，采用用 deflate 算法压缩数据；gzip 是一种流行的数据压缩算法，应用十分广泛，尤其是在 Linux 平台。

gzip 能力：当 Gzip 压缩到一个纯文本数据时，效果是非常明显的，大约可以减少 70%以上的数据大小。

gzip 作用：网络数据经过压缩后实际上降低了网络传输的字节数，最明显的好处就是可以加快网页加载的速度。网页加载速度加快的好处不言而喻，除了节省流量，改善用户的浏览体验外，另一个潜在的好处是 Gzip 与搜索引擎的抓取工具有着更好的关系。例如 Google 就可以通过直接读取 gzip 文件来比普通手工抓取更快地检索网页。

2 HTTP 协议中的压缩传输简介



第一：客户端向服务器请求头中带有：Accept-Encoding:gzip, deflate 字段，向服务器表示，客户端支持的压缩格式（gzip 或者 deflate)，如果不发送该消息头，服务器是不会压缩的。

第二：服务端在收到请求之后，如果发现请求头中含有 Accept-Encoding 字段，并且支持该类型的压缩，就对响应报文压缩之后返回给客户端，并且携带 Content-Encoding:gzip 消息头，表示响应报文是根据该格式压缩过的。

第三：客户端接收到响应之后，先判断是否有 Content-Encoding 消息头，如果有，按该格式解压报文。否则按正常报文处理。

3 在 Openfeign 技术中应用 GZIP 压缩

在 Spring Cloud 微服务体系中，一次请求的完整流程如下：

在整体流程中，如果使用 GZIP 压缩来传输数据，涉及到两次请求-应答。而这两次请求-应答的连接点是 Application Client，那么我们需要在 Application Client 中配置开启 GZIP 压缩，来实现压缩数据传输。

3.1 只配置 Openfeign 请求-应答中的 GZIP 压缩

只开启 Feign 请求-应答过程中的 GZIP，也就是浏览器-Application Client 之间的请求应答不开启 GZIP 压缩。

在全局配置文件中，使用下述配置来实现 Openfeign 请求-应答的 GZIP 压缩

```
server:
  port: 8082

spring:
  application:
    name: openfeign-client

eureka:
```

```yaml
  client:
    service-url:
      - http://localhost:8761/eureka/
feign:
  compression:
    request:
      # 开启请求 GZIP
      enabled: true
      # 设置支持 GZIP 压缩的 MIME 类型，即请求/响应类型。
      mime-types:
        - text/xml
        - application/xml
        - application/json
      # 配置启动压缩数据的最小阀值，单位字节。默认为 2048
      min-request-size: 512
    # 开启响应 GZIP
    response:
      enabled: true
```

3.2 配置全局 GZIP 压缩

在全局配置文件中配置下述内容，来开启所有请求-应答中的 GZIP 压缩，这里使用的是 Spring Boot 中的 GZIP 技术。在 Spring Boot 中已经集成了 GZIP 压缩技术，并对所有的请求-应答实现 GZIP 数据压缩。工程中已经依赖了 Spring Boot 技术，所以在配置文件中可以开启 Spring Boot 中的 GZIP 压缩技术，对完整流程中所有相关的请求-应答开启 GZIP 压缩。

```yaml
server:
  port: 8082
  compression:
    # 开启 GZIP
    enabled: true
    # 设置支持 GZIP 压缩的 MIME 类型，即请求/响应类型。
    mime-types:
      - application/json
      - application/xml
      - text/html
      - text/xml
      - text/plain
    # # 配置启动压缩数据的最小阀值，单位字节。默认为 2048
    min-response-size: 512

spring:
```

```yaml
  application:
    name: openfeign-client

eureka:
  client:
    service-url:
      - http://localhost:8761/eureka/
feign:
  compression:
    request:
      # 开启请求 GZIP
      enabled: true
      # 设置支持 GZIP 压缩的 MIME 类型，即请求/响应类型。
      mime-types:
        - text/xml
        - application/xml
        - application/json
      # 配置启动压缩数据的最小阀值，单位字节。默认为 2048
      min-request-size: 512
    # 开启响应 GZIP
    response:
      enabled: true
```

四、 配置 Openfeign 负载均衡请求超时时间

Openfeign 技术底层是通过 Ribbon 技术实现的，那么在负载均衡和超时时间配置上，主要对 Ribbon 的配置。具体配置如下：

1 超时时间配置

```yaml
ribbon:
  # 请求连接的超时时间，单位毫秒，默认的时间为 1 秒
  ConnectTimeout: 1000
  # 请求处理的超时时间，单位毫秒，默认的时间为 1 秒
  ReadTimeout: 1000
```

2 负载均衡配置

```yaml
# 设置负载均衡策略。openfeign-service 为设置负载均衡的服务名称
openfeign-service:
  ribbon:
    NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RandomRule
```